

P-Callup

Predicting Major League Performance from Triple-A, and Correcting for Who Gets Promoted
PM: Anish Awasthi · CMC'28 · 650-445-2131 · anishawasthi2@gmail.com

Project Overview

During each MLB season, front offices decide which Triple-A players are ready to be called up to play in the major league. This project aims to build a model to answer the question behind that decision: based on a player's performance in a AAA season, how will he perform in the MLB?

The intuitive approach is to have the model learn from players who were promoted, as we have both their AAA and MLB numbers. However, teams promote players they already believe will succeed using various scouting information and intangibles. So if you train a model only on promoted players, you are basing your prediction on a group pre-selected for success and will systematically overestimate everyone else.

In our data, 638 Triple-A player-seasons have usable MLB statistics and 2,529 do not, so that's 75% of the population. The fix is to use a technique pollsters use. If a certain group rarely answers the phone, you weigh each response you do get from them more.

Our missing group is players who never got called up. The question we can ask is: how likely was this player to be called up at all? So we then train our model *again*, this time weighing the unlikely call-ups much more heavily.

The project will run in three phases: build the "naive" model, measure how far off it is, then build the corrected version and compare.

Deliverables: An interactive Streamlit application with three views: a player card showing any AAA player's season alongside both the naive and corrected major-league projections, with the factors driving each; a side-by-side leaderboard showing which players the correction moves most, and in which direction. *If time*: A "Would you have called him up?", an interactive panel that shows visitors an anonymized stat line and asks them to guess whether that player was promoted. Also a written analysis quantifying the difference between the two models.

Datasets

Baseball Savant (Statcast). Pitch-level tracking data — for every pitch, the release velocity, spin rate, movement, and location; for every batted ball, the exit velocity, launch angle, and distance. This is the same camera-and-radar system that produces the numbers on MLB broadcasts. We have ingested and validated the full archive for both levels:

Level	Seasons	Game Dates	Pitches
MLB	2023-2025	552	2,145,111
AAA	2023-2025	457	2,609,577
Total		1009	4,754,688

MLB Stats API. Season stat lines, rosters, schedules, and the team-to-level mapping. This is what defines the cohort — who played at which level, how much, and in which season. Filtering to players with a meaningful Triple-A season yields 2,529 player-seasons, of which 638 have a usable major-league season the following year.

Goals

1. **Learn what features describe a hitter:** We have 4.75 million pitch-level records, can we use them to find out whether: exit-velocity distribution, hard hit %, chase rate, strikeout and walk rates, age relative to league matters the most.
2. **Estimate park factors:** Triple-A parks vary enormously — Albuquerque is at 5,100 feet, Reno at 4,500, Salt Lake at 4,200. Thin air makes the ball carry farther, so we need to account for that when comparison.
3. **Train the naive model:** Predict major-league wOBA from Triple-A features, using only the 638 players who were promoted and got a real MLB sample. Player train and test split so no player is in both.
4. **Model who gets promoted:** Using all 2,529 player-seasons, predict which players were called up. This will tell us what to correct and also what front offices value most.
5. **Measure the bias:** Compare the promoted and never-promoted groups feature by feature; compare the two models' projections on the same players; and check whether the naive model's accuracy degrades on the surprise call-ups, the promoted players who most resemble the never-promoted group.
6. **Build the corrected model and compare:** Retrain on the same promoted players, weighting each by how unlikely his promotion was. Quantify the difference and ship the interactive application.

Tech Stack

Python, with pandas and NumPy for data aggregation. Modeling: scikit-learn, Ridge regression for the translation model. Regularized linear regression - coefficients are interpretable and because our features are highly correlated with one another (max exit velocity, 90th-percentile exit velocity, and hard-hit rate all measure roughly the same thing), so ordinary regression coefficients will be unstable.

We will also compare gradient boosting to ridge. If gradient boosting barely beats ridge, the relationship is essentially linear and we prefer ridge for interpretability. For the promotion model, we'll use Logistic regression, since promotion is a yes/no outcome.

For visualization and UI: matplotlib and seaborn for analysis plots; plotly for interactive charts; SHAP to show which of a player's stats drove his projection on the player card. The application will be built in Streamlit and hosted free on Streamlit Community Cloud.

Git and Github for branches and pull requests.

Member Requirements

Recruiting 2–3 members. Roughly 4–6 hours per week, including one working session together. Comfort with python is great, familiarity with pandas helps as well. No ML knowledge is expected.

Planned Timeline

Week 1 (9/8) — Onboarding

Understand the setup, clone the repo, reproduce the existing numbers. Quick Baseball stats primer: what AAA is, what wOBA measures, why promotion is an interesting event. Each member walks the group through one module of the existing pipeline.

Verify: reproduce the 638 / 2,529 cohort split independently.

Week 2 — Features, part 1

Convert batting statistics from pitch-by-pitch data to season averages for each player per season. Verify by picking three players and confirming them with Baseball Savant's website by hand.

Week 3 — Features, part 2, and park factors

Finish the feature set (chase rate, age relative to league). Estimate each Triple-A park's offensive effect, shrinking toward the league average. Verify that Reno and Salt Lake, for example, should show up as inflating offense.

Week 4 — The naive model

Ridge and gradient boosting predicting major-league wOBA, trained on the 638 promoted players. Player-level train/test split. Make sure no player appears in both train and test, and compare against the predict-the-league-average baseline — if the model doesn't beat it, the features are the problem, not the model.

Week 5 — Who gets promoted

Logistic regression on all 2,529 player-seasons predicting call-up. Compare the promoted and never-promoted groups feature by feature. This will help us find out what front offices look for.

Week 6 — The correction, and first working app

Weight each promoted player by how unlikely his promotion was. Retrain. Compare coefficients against the naive model. Also, start working on the streamlit dashboard for the demo

Week 7 — Evaluation

Are promoted and never-promoted players actually different? We can compare their stats side by side. How much does the correction move things? Does the naive model's accuracy decrease for surprise call-ups?

Week 8 — The interactive application

Build out the rest of the dashboard and the interactive elements. Get ready to deploy publicly.

Week 9 — Writeup and presentation

Poster presentation and write-up